

GUIA DE ONBOARDING

QA & Automação de Testes

Manual introdutório para pessoas que estão iniciando no setor de QA do Projeto Atual Mais, com foco em teste manual e automação com Playwright.

ATUAL SOLUÇÕES

Material interno · Setor de QA · Versão 1.2

1. Objetivo do manual

Este manual apresenta a estrutura básica utilizada pelo setor de QA e Automação de Testes do Projeto Atual Mais: os ambientes de teste, as ferramentas do dia a dia, o fluxo de trabalho entre teste manual e automatizado, e a forma esperada de documentar e registrar as atividades.

Objetivo principal: dar autonomia rápida a quem está entrando no setor, reduzindo a dependência de explicações verbais e reunindo em um só lugar o necessário para começar a testar e automatizar com segurança.

Versão em mapa mental: este mesmo guia também está disponível em formato visual/interativo em mm.tt/map/4095504042.

2. Visão geral do setor de QA

O time de QA é responsável por validar as entregas do Atual Mais antes que cheguem à produção, cobrindo tanto testes manuais quanto testes automatizados end-to-end com Playwright.

O trabalho envolve: analisar cards e bugs reportados, desenhar cenários de teste, validar manualmente nos ambientes apropriados, automatizar os fluxos mais críticos e recorrentes, e documentar resultados de forma que qualquer pessoa do time consiga reproduzir o que foi encontrado.

O Atual Mais é um sistema ERP web (frontend React/Ant Design, via Umi) que cobre operações comerciais, fiscais, financeiras e administrativas — por isso, testes de fluxos fiscais e de venda exigem atenção redobrada.

3. Ferramentas utilizadas

As ferramentas abaixo fazem parte da rotina do setor de QA e possuem funções específicas no processo de teste e automação:

Ferramenta	Finalidade
Jira	Gestão dos cards de teste, bugs encontrados e acompanhamento das entregas do time de QA.
Clockify	Registro do tempo dedicado a cada atividade de teste, manual ou automatizado.
Clarity	Gerenciamento e controle de acessos utilizados nos ambientes de teste.
GitHub	Versionamento e colaboração sobre o código do projeto de automação de testes.
GitHub Actions	Pipeline de CI: roda a suíte de regressão em todo push/PR e a suíte completa todas as noites.
VS Code	Editor utilizado para escrever, rodar e depurar os testes automatizados.
Playwright	Framework de automação de testes end-to-end utilizado pelo time de QA.
JAM	Extensão do navegador para gravar o passo a passo e evidenciar bugs encontrados.
DevTools (navegador)	Inspeção de elementos, seletores e chamadas de rede durante teste manual e automação.
Claude	Apoio na análise de HTML/seletores, escrita de testes e documentação das atividades.

Importante: nem toda automação substitui teste manual. Fluxos críticos (fiscal, pagamento, estoque) devem ser validados manualmente antes de serem automatizados.

4. Ambientes de teste

HMG: <https://backoffice.hmg.atualmais.com.br/> — ambiente inicial de validação. Todo card deve ser testado aqui antes de qualquer outra etapa.

PREPROD: <https://backoffice.preprod.atualmais.com.br/> — utilizado quando o card impacta operações críticas: vendas, entradas, tributos, orçamentos, OS e rotinas fiscais.

Produção — Backoffice: <https://backoffice.atualmais.com.br/>

Link de teste em produção: <https://company8.atualmais.com.br/>

Regra prática: HMG é o ambiente padrão de validação e de execução dos testes automatizados. A suíte Playwright (local ou no CI) nunca deve rodar contra produção.

5. Dados para testes fiscais

Para testes fiscais, utilizar sempre o ambiente de homologação. Abaixo estão os dados de teste definidos para NFC-e:

Campo	Valor
Token/ID	000001
Token/CSC	A8BEE0500FF3601DE0531E5FA00A9D26
Razão Social	SG AUTOMAÇÃO COMERCIAL LTDA ME - ATUAL SGP
CNPJ	12.266.496/0001-29
IE	083.632.80-8

- Sempre realizar os testes fiscais em homologação.
- Utilizar séries altas para testes, pois séries baixas podem estar sendo usadas por outras pessoas.
- O certificado digital deve ser solicitado a um integrante do time.
- Mesmo tendo Token/ID e Token/CSC, o certificado digital continua sendo necessário para determinados testes fiscais.
- Esses mesmos dados devem ser usados como massa de teste em cenários automatizados que envolvem NFC-e.

6. Fluxo de trabalho: teste manual e automação

- Receber e entender o card no Jira (bug, melhoria ou nova funcionalidade).
- Mapear os cenários de teste: caminho feliz, casos de borda e combinações relevantes.
- Validar manualmente no ambiente HMG (e em PREPROD quando aplicável).
- Registrar o tempo da atividade no Clockify.
- Avaliar se o cenário é candidato à automação (fluxo crítico, recorrente ou propenso a regressão).
- Quando aplicável, escrever ou atualizar o teste automatizado em Playwright, em **tests/cards/** com a tag do card.
- Rodar a suíte de testes localmente antes de subir o código.
- Documentar evidências, comportamento encontrado e resultado do teste no card.
- Se o cenário for crítico/recorrente, promover o teste para **tests/regression/** depois que o card for validado.
- Atualizar o card no Jira com o status e as informações necessárias.

7. Estrutura do projeto de automação (Playwright)

O código de automação vive no repositório github.com/romuloatual/atal_mais. A estrutura segue o padrão do Playwright Test, organizada em duas pastas com convenções de tag diferentes:

- **playwright.config.ts** — configuração global (baseURL padrão = HMG, navegadores, timeouts, relatórios).
- **tests/regression/** — telas/fluxos mais usados. Toda spec aqui usa a tag **@regression**. É a suíte que roda sempre (todo push/PR e à noite), pegando quando um card quebra outra funcionalidade do mesmo processo/tela.
- **tests/cards/** — um arquivo por card do Jira (ex.: **INTG-2645.spec.ts**), com a tag do card (ex.: **@INTG-2645**). Roda sob demanda enquanto o card está em andamento.
- **package.json** — scripts npm do projeto.

Scripts npm principais:

<code>npm test</code>	- roda toda a suíte
<code>npm run test:headed</code>	- roda os testes com o navegador visível
<code>npm run test:ui</code>	- abre a interface interativa do Playwright
<code>npm run test:report</code>	- abre o último relatório de execução
<code>npm run test:codegen</code>	- grava ações no navegador e gera código de teste automaticamente
<code>npm run test:regression</code>	- roda só a suíte de regressão (tag <code>@regression</code>)
<code>npm run test:card -- @INTG-2645</code>	- roda só os testes de um card específico

Antes de escrever um teste novo, rode **npm run test:codegen** no fluxo que você quer automatizar — ele já gera boa parte dos seletores e das ações.

8. Integração contínua (CI)

O workflow `.github/workflows/playwright.yml` automatiza a execução dos testes no GitHub Actions:

- **Push/PR para main:** roda só a suíte de regressão (*@regression*) — feedback rápido sobre se a alteração quebrou algo em outra parte do mesmo processo.
- **Todo dia às 03:00 (horário de Brasília):** roda a suíte completa (regressão + cards), agendado (*schedule*), para pegar qualquer drift que passou despercebido.
- **Disparo manual (*workflow_dispatch*):** permite escolher entre rodar só a regressão ou a suíte completa quando necessário.
- O relatório HTML de cada execução fica disponível como artefato do workflow por 14 dias, e também publicado (sempre a versão mais recente) em romuloatual.github.io/atual_mais via GitHub Pages.

A CI nunca aponta para produção — o baseURL padrão do projeto é o ambiente HMG, o mesmo usado nas validações manuais.

9. Boas práticas para seletores e testes automatizados

- Prefira seletores estáveis: id (*#id*), role/texto visível (*getByRole*, *getByText*) e atributos *data-testid* quando existirem.
- Evite classes do CSS Modules com hash (ex.: *customerName__AxNXm*) — esse hash muda a cada build/deploy e quebra o teste sem que a funcionalidade tenha mudado.
- Ao mapear uma tela nova, use o DevTools (Copy > Copy outerHTML) para capturar o HTML já renderizado — o app é uma SPA (Umi/React), então o HTML estático da página não é suficiente.
- Para telas dentro de modais, popups ou iframes (ex.: pré-visualização de impressão), confirme se o elemento está dentro de um *#document* aninhado antes de tentar mapear os seletores.
- Documente qualquer inconsistência entre o esperado no card e o que a tela realmente mostra — isso pode indicar que o bug já foi corrigido parcialmente ou que o escopo mudou.

10. Como documentar um problema

Ao identificar um erro, o registro deve permitir que outra pessoa consiga reproduzir e compreender o problema sem depender apenas de explicações verbais.

- Descrever de forma objetiva o que estava sendo realizado.
- Informar o caminho utilizado até chegar ao erro (módulo, tela, fluxo).
- Registrar o comportamento esperado e o comportamento apresentado.
- Informar o ambiente em que o problema ocorreu (HMG, PREPROD, Produção).
- Adicionar evidências relevantes (prints, HTML, vídeo).
- Quando necessário, usar o JAM para gravar o passo a passo e o momento exato do erro.
- Em testes automatizados, anexar o relatório do Playwright (screenshot, trace e vídeo da falha) ao card — ou linkar o artefato publicado pela CI no GitHub Actions.

11. Checklist de entrada no setor de QA

- ☐ Acessar e conhecer o Jira do projeto.
- ☐ Configurar e utilizar o Clockify.
- ☐ Ter acesso ao Clarity.
- ☐ Ter acesso ao GitHub e clonar o repositório github.com/romuloatual/atual_mais.
- ☐ Ter o VS Code configurado, com a extensão Playwright Test for VSCode instalada.
- ☐ Instalar o Node.js e rodar `npm install` no projeto de automação.
- ☐ Rodar `npx playwright install` para instalar os navegadores de teste.
- ☐ Instalar/conhecer o JAM.
- ☐ Conhecer o uso do Claude como ferramenta de apoio.
- ☐ Conhecer os ambientes HMG, PREPROD e Produção.
- ☐ Conhecer a convenção de tags (regression/cards) e a pipeline de CI no GitHub Actions.
- ☐ Solicitar acesso/certificado digital quando necessário para testes fiscais.
- ☐ Rodar a suíte de testes local pela primeira vez (`npm test`) e confirmar que passa.

12. Principais regras

- Todo teste, manual ou automatizado, deve estar vinculado a um card no Jira.
- O tempo das atividades deve ser registrado no Clockify.
- Testes automatizados (local ou CI) rodam por padrão contra HMG — nunca contra produção.
- Todo teste novo entra em **tests/cards/** com a tag do card; só é promovido para **tests/regression/** depois de validado e considerado crítico/recorrente.
- Seletores baseados em classes com hash (CSS Modules) devem ser evitados na automação.
- Erros devem ser documentados de forma clara e reproduzível, com evidências anexadas.
- Dados de teste fiscal devem ser usados somente nos ambientes apropriados.
- Materiais e documentos do setor devem seguir o padrão visual da Atual Soluções.

Mentalidade esperada: entender o fluxo antes de testar, validar manualmente antes de automatizar, e deixar evidências suficientes para que qualquer pessoa do time consiga reproduzir o que foi encontrado.

13. Identidade Visual — Atual Soluções

A identidade visual dos materiais do setor deve seguir o padrão institucional da Atual Soluções, mantendo consistência nas apresentações, documentos e comunicações internas.

Elemento	Especificação
Verde institucional	PANTONE Green C · CMYK 100/0/65/0 · RGB 0/168/134 · HEX #00A886
Cinza institucional	PANTONE Cool Gray 10 C · CMYK 0/0/0/72 · RGB 102/102/102 · HEX #666666
Tipografia	Família sem serifa, com hierarquia clara entre títulos, subtítulos e texto.
Estilo	Visual limpo, corporativo e tecnológico, com boa hierarquia de informação e uso consistente das cores institucionais.

Regra: as cores institucionais não devem ser alteradas arbitrariamente em materiais oficiais.